CSL7530 : Advanced Machine Learning

Assignment-1 : Boosted Decision Trees for Regression and Classification

Krishna Balaji Patil (B22CS078)

This report presents the implementation details and experimental analysis of Gradient Boosted Decision Trees for both regression and classification tasks. The codebase consists of a custom implementation of regression tree used as weak learners within a gradient boosting framework. For regression, the model optimizes mean squared error, while for classification, multinomial deviance (cross-entropy loss) is minimized using a softmax formulation. Learning rate 'lr' is introduced to control the contribution of each weak learner, which improves training stability, prevents overfitting, and allows finer control over convergence even though the basic algorithm does not explicitly include it. Various hyperparameters including number of boosting rounds, learning rate and tree depth are systematically evaluated. The results are analyzed through training and testing performance metrics along with visualizations to study convergence behavior and generalization performance.

Code can be found at: GitHub Repository.

Code Structure Summary:

- `RegressionTree.py` : Contains the custom implementation of the Regression Tree used as the base learner.
- `GradientBoostingRegression.py` : Implementation of Algorithm 10.3 for Gradient Boosting in regression.
- `GradientBoostingClassification.py` : Implementation of Algorithm 10.4 for Gradient Boosting in multi-class classification.
- `california_housing_regression.py` : Script for evaluating the regression model on the California Housing dataset.
- `iris_classification.py` : Script for testing the classification model on the Iris dataset.
- `digits_classification.py` : Script for testing the classification model on the Digits dataset.
- `experiments.py` : Contains functions for hyperparameter analysis and generating performance plots.

1. Regression

For the regression task, I implemented a Decision Tree Regressor that recursively partitions the feature space by selecting splits that minimize the Mean Squared Error (MSE). At each node, the algorithm evaluates candidate thresholds and chooses the split that maximizes the reduction in variance:

$$\text{MSE} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \bar{y})^2.$$

The optimal split is selected by minimizing the weighted MSE of the left and right child nodes. Since decision trees are non-parametric and built via greedy recursive partitioning, there is no gradient computation or parameter initialization. The final prediction at each leaf node is the mean of the training targets in that region.

Hyper-parameters involved include:

- No. of iterations (M) : default value = 100
- Max depth of trees : default value = 3
- learning rate : default value = 0.05

For evaluating our regression algorithm, we used the **California Housing dataset**, which contains housing-related features such as median income, average rooms, population, latitude, and longitude, with the target variable being the median house value of districts in California. The dataset consists of 20,640 samples with 8 numerical input features. Before training, the data was randomly shuffled and split into 80% training data and 20% testing data. The training set was used to fit the regression model, while the held-out test set was used exclusively to evaluate generalization performance and report metrics such as Mean Squared Error. After running the Algorithm 10.3 with the above given hyperparameters, mse reported was as follows:

for training set : 0.30950573240260953

for test set : 0.33577755136884824

Hyperparameters analysis:

1. No. of iterations (M)

The Regression algorithm was run on the California Housing dataset for different values of M keeping other parameters constant, here is how the mse varies with M:

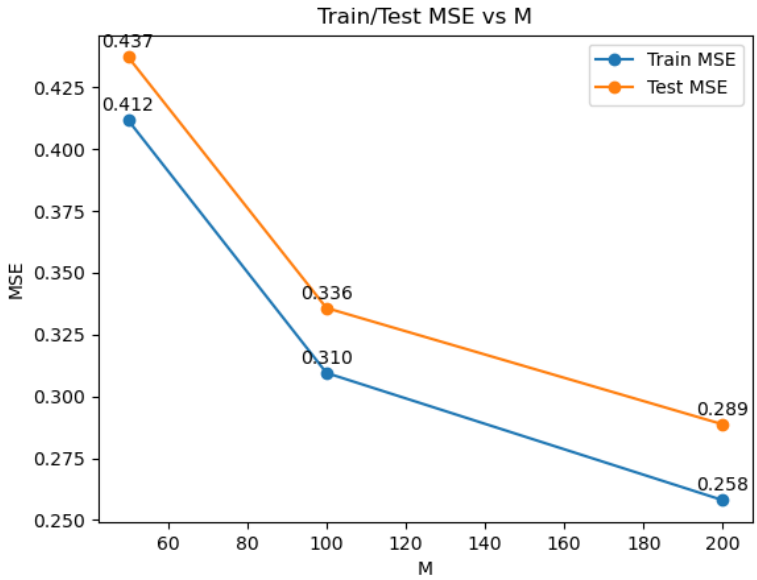

Figure 1: Train and Test MSE vs M

The mse values for every 5 boosting rounds were recorded for M=300 and here is the plot of mse vs M:

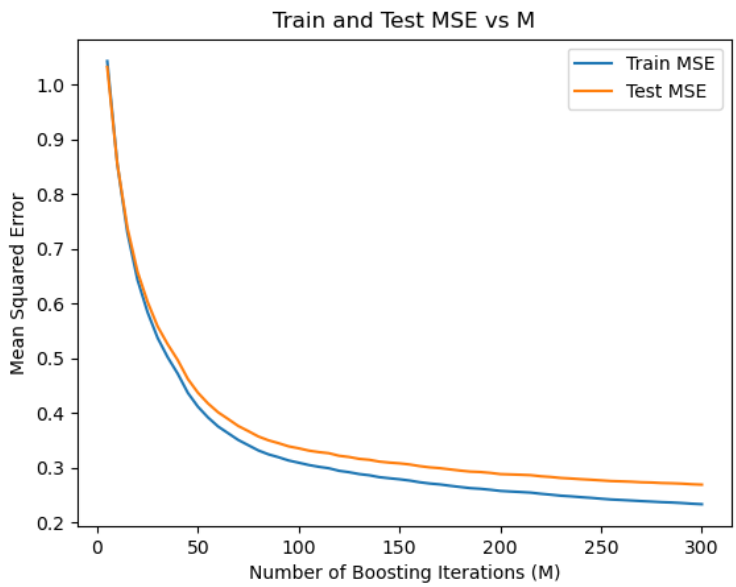

Figure 2: Training loss vs M (iterations)

2. Learning Rate (lr)

The regression algorithm was evaluated for different values of the learning rate lr while keeping the number of iterations M and tree depth constant. The learning rate controls the contribution of each weak learner to the final ensemble model.

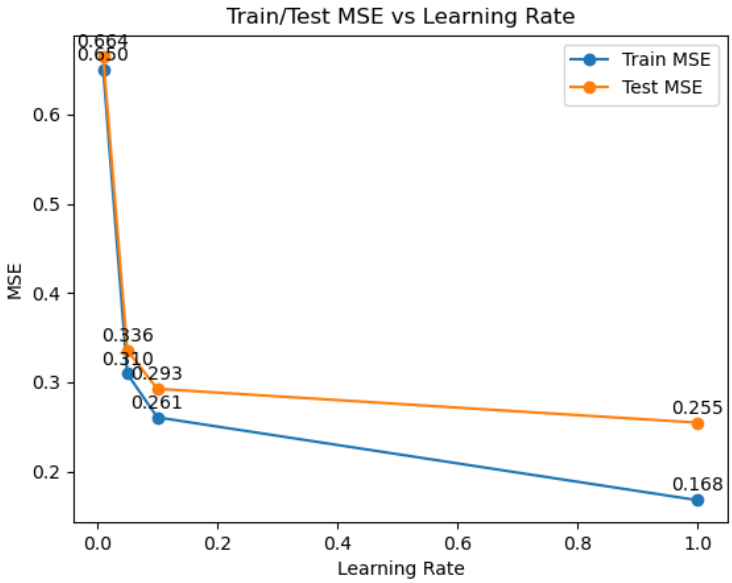

Figure 3: Train and Test MSE vs Learning Rate

3. Maximum Tree Depth

The model was also evaluated for varying maximum tree depths while keeping M and η constant. The tree depth determines the complexity of each weak learner and directly affects the model's capacity.

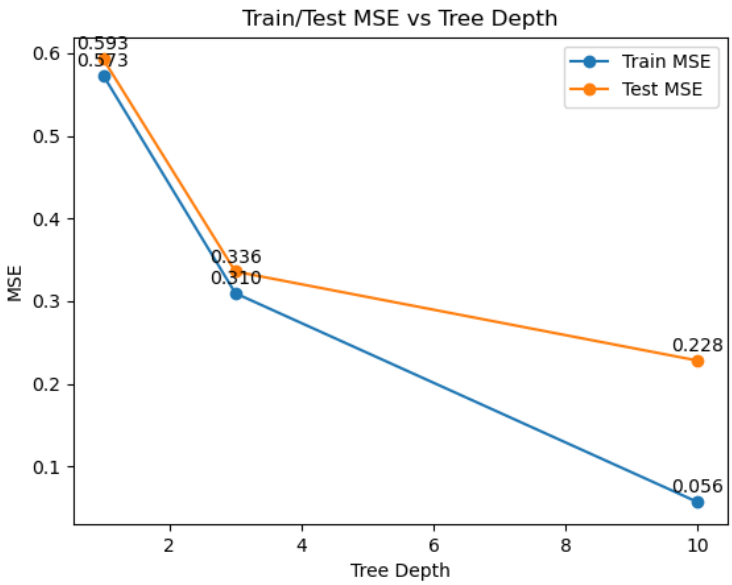

Figure 4: Train and Test MSE vs Tree Depth

Inference:

- Increasing the number of iterations M consistently decreases the training MSE,

since more weak learners are added to reduce residual errors. However, after a certain value of M, the test MSE stabilizes or slightly increases, indicating overfitting.

- A smaller learning rate lr leads to slower convergence but more stable and smoother reduction in training loss. It typically requires a larger number of iterations to achieve optimal performance.
- A larger learning rate accelerates convergence but may cause fluctuations in training and test error. After an optimal value, the rate hardly changes the test MSE.
- Increasing the maximum tree depth improves training performance because deeper trees capture more complex nonlinear patterns.
- However, larger tree depths increase model variance and can cause overfitting, resulting in higher test MSE. In the above graph, we see large drop in train MSE for max depth = 10 compared to max depth = 3, while the drop in test MSE was relatively much lower. This points towards overfitting.
- Optimal performance is achieved by balancing M, lr, and tree depth to control the bias–variance tradeoff and ensure good generalization on unseen data.

2. Classification

For the classification task, I implemented Gradient Boosting with decision trees as weak learners. The model is built in an additive manner, where at each iteration a shallow decision tree is trained to fit the negative gradient of the loss function. For multiclass classification, the loss function used was the multinomial deviance (softmax cross-entropy). At each boosting step, pseudo-residuals are computed as:

$$r_{ik} = y_{ik} - p_{ik},$$

where p_{ik} represents the predicted probability for class k. Each new tree is trained on these residuals, and the ensemble model is updated using the learning rate lr. The final prediction is obtained by selecting the class with the highest predicted probability.

Hyper-parameters involved include:

- No. of iterations (M) : default value = 20
- Max depth of trees : default value = 3
- learning rate : default value = 0.1

For evaluation, the algorithm was tested on both the **Iris dataset** and the **Digits dataset**. The Iris dataset contains 150 samples with 4 numerical features across 3 classes, while the Digits dataset consists of 1,797 samples of 8×8 grayscale images (64 features) across 10 classes (digits 0–9).

After running Algorithm 10.4 with the above hyperparameters, the accuracy reported was as follows:

for test set (Iris dataset) : 0.9666666666666667

for test set (Digits dataset) : 0.9055555555555556

For performance analysis and hyperparameter tuning, the Digits dataset was used due to its higher dimensionality and multi-class complexity. The dataset was randomly shuffled and split into 80% training data and 20% testing data. The training set was used to build the ensemble model, and the test set was used to evaluate classification accuracy.

Hyperparameters analysis:

1. No. of iterations (M)

The classification algorithm was run on the Digits dataset for different values of M while keeping other parameters constant. The variation of training and test accuracy with respect to M is shown below:

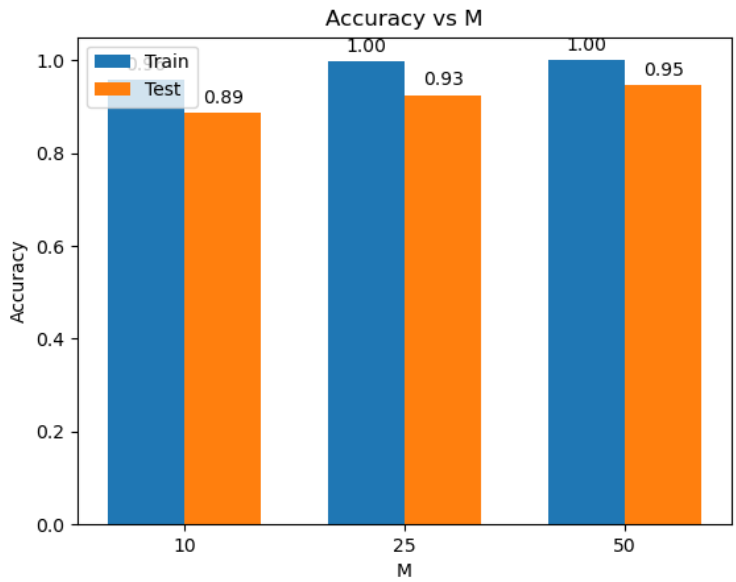

Figure 5: Train and Test Accuracy vs M

The accuracy values across iterations were also recorded for a larger value of M to analyze convergence behaviour:

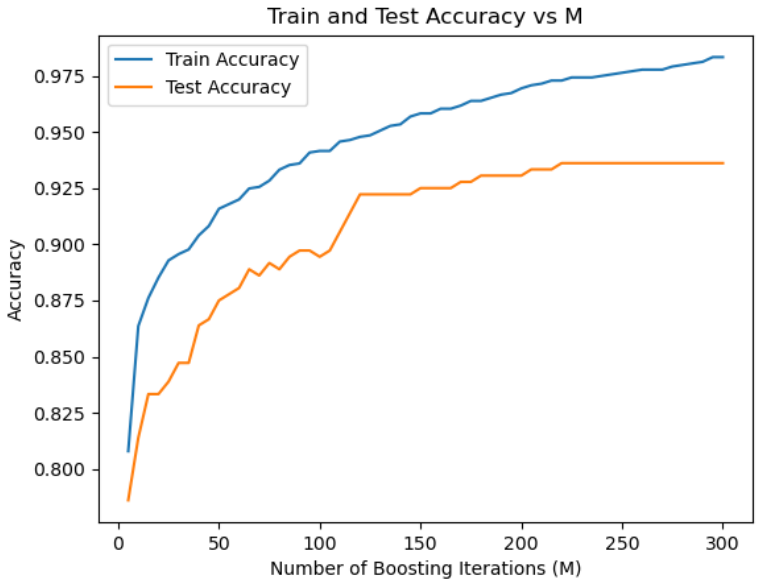

Figure 6: Training Accuracy vs M (iterations)

2. Learning Rate (lr)

The model was evaluated for different learning rates while keeping M and tree depth constant. The learning rate determines how much each tree contributes to correcting previous classification errors.

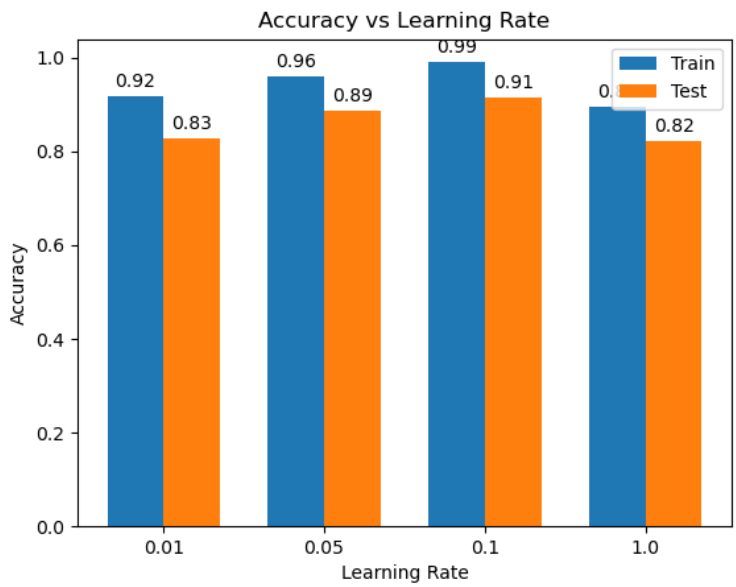

Figure 7: Train and Test Accuracy vs Learning Rate

3. Maximum Tree Depth

The effect of varying maximum tree depth was also studied while keeping other parameters fixed. Tree depth controls the complexity of each weak learner and directly affects the

classifier's capacity.

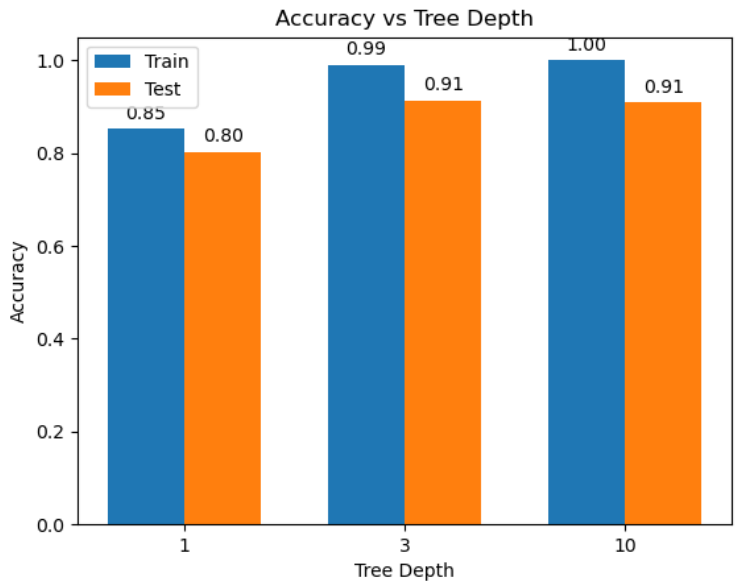

Figure 8: Train and Test Accuracy vs Tree Depth

Inference:

- Increasing the number of iterations M generally improves training accuracy, as more weak learners are added to reduce classification errors. However, beyond a certain point, test accuracy may plateau (as observed in above graph) or slightly decrease due to overfitting.
- A smaller learning rate lr results in slower but more stable improvement in accuracy, often requiring a larger number of iterations for optimal performance.
- A larger learning rate speeds up convergence but may lead to instability or reduced test accuracy if too aggressive. In above experiment, we can see how the accuracy for both train and test data drops for lr = 1.0 in comparison to lr = 0.05 or lr = 0.1.
- Increasing tree depth improves training accuracy by enabling more complex decision boundaries.
- However, deeper trees can increase variance and reduce test accuracy due to overfitting. Here we see that for depth = 3 and depth = 10, the model gives same test accuracy while it overfits to the train data with an accuracy of 1 for depth = 10.
- Proper tuning of M, lr, and tree depth is necessary to balance bias and variance and achieve strong generalization on unseen data.